

Rapport d'Audit Exhaustif — Java Legacy Modernizer v4.0

Auteur : Hamza NORDINE **Date** : 8 avril 2026 **Version** : 4.0.0 (checkpoint 74fa228c)

Plateforme : EJB Client Modernizer + Compleo v1.0

1. Synthese Executif

La plateforme **Java Legacy Modernizer v4.0** est une application web full-stack (React 19 + Express 4 + MySQL) conçue pour analyser, transformer et moderniser des projets Java legacy vers des architectures Spring Boot / Cloud-Native. Elle se compose de deux moteurs principaux : un **analyseur multi-technologies** (côté client, 83 règles IA) et un **moteur Compleo** (côté serveur, parsing AST + génération Spring Boot). Le projet totalise **29 970 lignes de code source** réparties sur **123 fichiers**, avec **119 tests vittest** passant en 791ms.

L'application est fonctionnelle et couvre un périmètre large. Cependant, l'audit révèle des axes d'amélioration significatifs en matière de sécurité (absence d'authentification sur les procédures tRPC), de maintenabilité (fichiers "God Object" dépassant 1 000 lignes), et de robustesse (stockage Compleo en mémoire volatile). Ce rapport détaille l'état des lieux complet avec des recommandations priorisées.

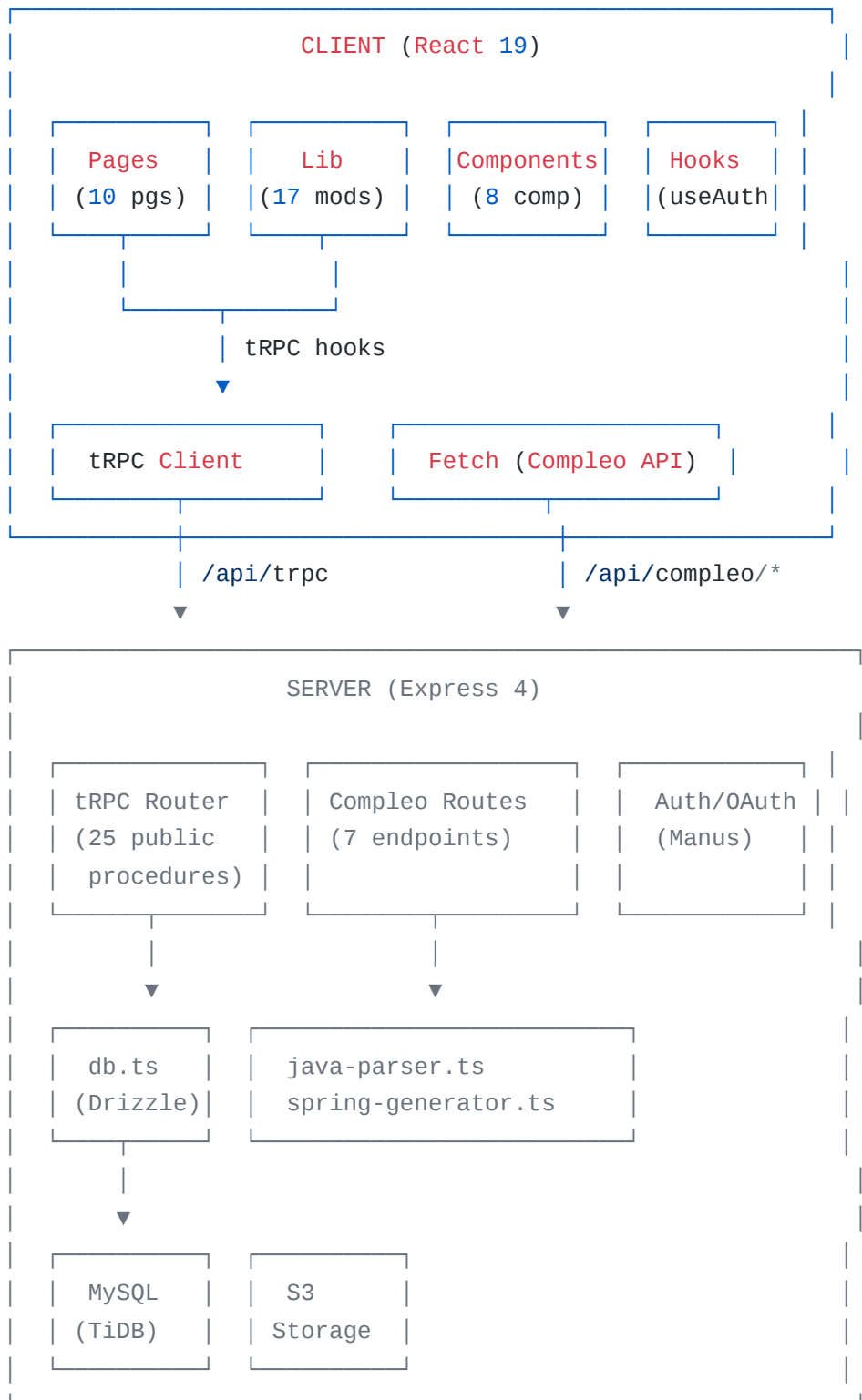
2. Architecture Technique

2.1 Stack Technologique

Couche	Technologie	Version
Frontend	React + TypeScript	19.2.1 + 5.9.3
Styling	Tailwind CSS + shadcn/ui	4.1.14
Routing client	Wouter	3.3.5
State management	TanStack React Query + tRPC	5.90.2 + 11.6.0
Animations	Framer Motion	12.23.22
Graphes	Cytoscape.js	3.31.x
Charts	Recharts	2.15.2
Backend	Express + tRPC	4.21.2 + 11.6.0
Base de données	MySQL (TiDB) + Drizzle ORM	0.44.5
Stockage fichiers	AWS S3	3.693.0
Build	Vite	7.1.7
Tests	Vitest	2.1.9
Sérialisation	SuperJSON	1.13.3

2.2 Architecture Applicative

L'application suit une architecture **monolithique modulaire** avec séparation client/serveur :



2.3 Schema de Base de Données

La base de données comprend **7 tables** (+ 1 table `users` système) :

Table	Colonnes	Description
users	8	Utilisateurs OAuth (id, openId, name, email, role, etc.)
projects	15	Projets d'analyse (nom, description, technologies, scores, git)
project_files	7	Fichiers Java uploadés par projet
scans	21	Résultats d'analyse (scores, issues, microservices, cloud, AI, migration)
comments	10	Commentaires sur les analyses (auteur, type, statut validation)
git_connections	9	Connexions Git (GitHub, GitLab, Bitbucket, Azure DevOps)
shared_reports	7	Rapports partagés via lien public (token unique)

3. Inventaire des Fonctionnalités

3.1 Moteur d'Analyse Multi-Technologies (côté client)

Le moteur d'analyse fonctionne entièrement côté client (navigateur) et comprend 4 sous-modules :

3.1.1 Analyseur EJB (`ejb-analyzer.ts` — 598 lignes)

- Détection des injections EJB (`@EJB` , `@Inject` , `@Resource`)
- Détection des lookups JNDI (`InitialContext` , `lookup()`)
- Extraction des appels de méthodes et types de retour
- Détection des transactions (`@TransactionAttribute`)
- Détection des éléments JMS (`@MessageDriven` , `MessageListener`)
- Construction du graphe de dépendances inter-classes

3.1.2 Analyseur Legacy Multi-Technologies (`legacy-analyzer.ts` — 1 204 lignes)

Technologie	Patterns détectés
EJB	@Stateless , @Stateful , @Singleton , @MessageDriven , @Entity
Servlet	HttpServlet , doGet/doPost , @WebServlet , ServletContext
JSP	<jsp: , <% , scriptlets, taglibs
Struts	Action , ActionForm , struts-config.xml , ActionMapping
SOAP	@WebService , @WebMethod , SOAPMessage , WSDL
JDBC	DriverManager , PreparedStatement , ResultSet , Connection
Hibernate	SessionFactory , HQL , Criteria , @Entity
JMS	ConnectionFactory , MessageProducer , @JMSListener
Batch	@BatchProperty , ItemReader , ItemWriter , JobOperator
JPA	EntityManager , @PersistenceContext , CriteriaBuilder

3.1.3 Moteur IA — 83 Règles (ai-engine.ts — 1 594 lignes)

Catégorie	Nombre de règles	Sévérités
Sécurité (OWASP)	12	8 critical, 4 warning
SQL Injection avancée	5	5 critical
Anti-patterns	8	2 critical, 6 warning
Qualité de code	10	1 critical, 9 info
Performance	5	5 warning
Couplage	4	4 warning
Transactions	5	3 critical, 2 warning
Concurrence	4	4 critical
Sérialisation	2	2 warning
Logging	2	1 critical, 1 info
Architecture	3	3 warning
Error Handling	2	2 warning
JPA/Hibernate	2	2 warning
Modernisation	19	Suggestions
Total	83	22 critical, 32 warning, 10 info, 19 suggestions

Le scoring utilise un système pondéré : critical (-5), warning (-2), info (-1), avec un score de base de 100.

3.1.4 Extracteur de Microservices (`microservice-extractor.ts` — 641 lignes)

- Extraction DDD (Domain-Driven Design) basée sur les packages Java
- Identification des bounded contexts
- Proposition de microservices avec APIs REST, événements, et bases de données
- Estimation de complexité et effort de migration

3.2 Générateurs de Code (côté client)

3.2.1 Générateur de Code Modernisé (`code-generator.ts` — 710 lignes)

- Transformation EJB → Spring Boot (annotations, injection CDI → Spring DI)
- Génération de Controllers REST à partir des interfaces Remote
- Génération de Services avec `@Transactional`

3.2.2 Générateur Étendu (`extended-generator.ts` — 1 619 lignes)

Transformation	Entrée	Sortie
Servlet → REST	<code>HttpServlet</code>	Spring <code>@RestController</code>
Struts → Spring MVC	<code>Action</code> , <code>ActionForm</code>	<code>@Controller</code> , DTOs
SOAP → REST	<code>@WebService</code>	<code>@RestController</code> + OpenAPI
JDBC → JPA	<code>PreparedStatement</code>	Spring Data JPA Repository
Hibernate → JPA	<code>SessionFactory</code> , HQL	Spring Data JPA + Specifications
JMS → Kafka	<code>MessageDriven</code>	<code>@KafkaListener</code> + <code>KafkaTemplate</code>
Batch → Spring Batch	<code>ItemReader/Writer</code>	<code>@StepScope</code> + <code>Job</code>
EJB → Spring	<code>@Stateless/@Stateful</code>	<code>@Service</code> + <code>@Transactional</code>

3.2.3 Générateur Cloud-Native (`cloud-generator.ts` — 910 lignes)

- Dockerfile multi-stage (build + runtime)
- Kubernetes manifests (Deployment, Service, ConfigMap, HPA)
- Helm Charts complets
- API Gateway (Spring Cloud Gateway)
- Observabilité (Prometheus, Grafana, Jaeger)
- Docker Compose
- GitHub Actions CI/CD
- Spring Security config

3.3 Moteur Compleo v1.0 (côté serveur)

Le moteur Compleo est un pipeline complet EJB → Spring Boot fonctionnant côté serveur :

3.3.1 Parser Java (`java-parser.ts` — 906 lignes, 28 fonctions)

- Parsing regex/heuristique spécialisé pour le pattern BOA EAI
- Détection : UseCases (BaseUseCase), DTOs (VoIn/VoOut), Services, Enums, Exceptions, Validators, Remote Interfaces, Base Classes, Constants
- Extraction des champs DTO avec résolution de types (String, Integer, Long, BigDecimal, Date, Boolean, List, etc.)
- Extraction des injections (`@Inject` , `@EJB` , `@Resource` , `@Autowired`)
- Extraction des informations transactionnelles
- Parsing du pom.xml Maven (groupId, artifactId, version, dépendances)
- Sortie : IR (Intermediate Representation) JSON structuré

3.3.2 Générateur Spring Boot (`spring-generator.ts` — 1 184 lignes, 25 fonctions)

Artefact généré	Description
*Controller.java	REST Controller par domaine (Lombok, Slf4j, Jakarta Validation)
*Service.java	Service par domaine (@Transactional, injection constructeur)
*RequestDTO.java	DTO de requête (Lombok @Data, Jakarta @NotNull/@Size)
*ResponseDTO.java	DTO de réponse (Lombok @Data)
*Enum.java	Enums préservés
*Exception.java	Exceptions métier
GlobalExceptionHandler.java	@ControllerAdvice centralisé
*Validator.java	Validators Spring (@Component)
*ServiceAdapter.java	Adaptateurs pour interfaces Remote (WebClient)
*ControllerTest.java	Tests MockMvc (@WebMvcTest)
Application.java	Main class Spring Boot
application.yml	Configuration (DB, JPA, logging, actuator)
Dockerfile	Multi-stage build
docker-compose.yml	Stack complète (app + MySQL)
k8s-deployment.yml	Deployment + Service Kubernetes
pom.xml	Spring Boot 3.2 + dépendances
MIGRATION_REPORT.md	Rapport de migration détaillé

Résultats sur les 3 projets EJB fournis :

Projet	Fichiers Java	UseCases	DTOs	Fichiers générés	Lignes générées
boa-acl-test-complet	56	8	18	47	1 495
boa-realistic-ejb-project	68	11	27	78	2 279
boa-ultimate-test	78	12	28	64	2 041

3.4 Fonctionnalités Transversales

3.4.1 Web Workers — Analyse Parallèle

- `analysis-worker.ts` (379 lignes) : Worker autonome avec les 3 moteurs d'analyse
- `worker-pool.ts` (550 lignes) : Orchestrateur multi-workers avec chunking adaptatif
- Détection automatique du nombre de cœurs CPU (`navigator.hardwareConcurrency`)
- Seuil d'activation : 10+ fichiers Java
- Barre de progression en temps réel avec ETA, vitesse (fichiers/s), technologies détectées

3.4.2 Export de Rapports

- Export PDF du rapport d'analyse IA (jsPDF — 670 lignes)
- Export PDF du plan de migration Strangler Fig (635 lignes)
- Export Excel du plan de migration (6 feuilles — 253 lignes)
- Export ZIP du code généré (214 lignes)

3.4.3 Gestion de Projets (tRPC + MySQL)

- CRUD complet de projets avec métadonnées (technologies, scores, git)
- Upload et stockage de fichiers Java par projet
- Historique des scans avec résultats complets (JSON)
- Commentaires sur les analyses (avec workflow de validation)

- Connexions Git (4 providers : GitHub, GitLab, Bitbucket, Azure DevOps)
 - Partage de rapports via liens publics (token unique)
-

4. Pages de l'Application

Route	Page	Lignes	État	Description
/	Home	224	Fonctionnel	Landing page avec 6 cartes de fonctionnalités
/projects	Projects	342	Fonctionnel	Liste des projets, création, recherche
/projects/:id	ProjectDetail	1 360	Fonctionnel	Workspace d'analyse complet (éditeur, résultats, microservices, cloud)
/architecture/:id	Architecture	675	Fonctionnel	Graphe interactif Cytoscape.js (14 nœuds, 19 liens)
/migration/:id	Migration	409	Fonctionnel	Simulateur Strangler Fig avec export PDF/Excel
/collaboration/:id	Collaboration	313	Fonctionnel	Commentaires, validation, partage
/compleo	Compleo	921	Fonctionnel	Pipeline EJB → Spring Boot (upload, analyse, génération, preview)
/api-docs	ApiDocs	352	Fonctionnel	Documentation API REST (12 endpoints)
/component-showcase	ComponentShowcase	1 437	Fonctionnel	Galerie de composants shadcn/ui
/404	NotFound	52	Fonctionnel	Page 404

5. API Backend

5.1 Procédures tRPC (25 procédures publiques)

Namespace	Procédure	Type	Description
auth	me	query	Utilisateur courant
auth	logout	mutation	Déconnexion
projects	list	query	Liste des projets
projects	create	mutation	Créer un projet
projects	getById	query	Détail d'un projet
projects	update	mutation	Mettre à jour un projet
projects	delete	mutation	Supprimer un projet
files	upload	mutation	Uploader un fichier Java
files	list	query	Lister les fichiers d'un projet
files	deleteAll	mutation	Supprimer tous les fichiers
scans	create	mutation	Créer un scan
scans	list	query	Lister les scans d'un projet
scans	getById	query	Détail d'un scan
scans	updateResults	mutation	Mettre à jour les résultats
comments	create	mutation	Ajouter un commentaire
comments	list	query	Lister les commentaires
comments	validate	mutation	Valider un commentaire
comments	delete	mutation	Supprimer un commentaire
git	connect	mutation	Connecter un repo Git

Namespace	Procédure	Type	Description
git	list	query	Lister les connexions Git
git	disconnect	mutation	Déconnecter un repo
sharing	create	mutation	Créer un lien de partage
sharing	list	query	Lister les partages
sharing	getByToken	query	Accéder via token
system	notifyOwner	mutation	Notification propriétaire

5.2 API Compleo (Express REST — 7 endpoints)

Méthode	Endpoint	Description
POST	/api/compleo/upload	Upload ZIP (multer, max 50 Mo)
POST	/api/compleo/analyze	Analyser le projet EJB
POST	/api/compleo/generate	Générer le projet Spring Boot
GET	/api/compleo/download/:sessionId	Télécharger le ZIP généré
GET	/api/compleo/preview/:sessionId/*	Prévisualiser un fichier
GET	/api/compleo/session/:sessionId	Détail d'une session
GET	/api/compleo/sessions	Lister toutes les sessions

6. Tests

6.1 Résultats

Suite de tests	Tests	Assertions	Durée	Lignes
auth.logout.test.ts	1	4	5ms	62
routers.test.ts	32	56	22ms	530
compleo.test.ts	39	94	35ms	509
migration-export.test.ts	17	34	8ms	269
worker-pool.test.ts	30	55	14ms	340
Total	119	243	85ms	1 710

6.2 Couverture par Module

Module	Couvert par tests	Commentaire
tRPC Routers (projects, files, scans, comments, git, sharing)	Oui (32 tests)	Mocks DB, couverture CRUD complète
Java Parser	Oui (20 tests)	Parsing UseCases, DTOs, Services, Enums
Spring Generator	Oui (19 tests)	Controllers, Services, DTOs, Tests, Cloud
Worker Pool	Oui (30 tests)	Chunking, distribution, progression, annulation
Migration Export	Oui (17 tests)	PDF structure, Excel feuilles
Auth Logout	Oui (1 test)	Cookie clearing
Modules NON testés		
AI Engine (83 règles)	Non	Aucun test unitaire pour les règles
Legacy Analyzer	Non	Aucun test pour la détection de technologies
EJB Analyzer	Non	Aucun test pour l'analyse EJB
Extended Generator	Non	Aucun test pour les transformations
Cloud Generator	Non	Aucun test pour la génération cloud
Microservice Extractor	Non	Aucun test pour l'extraction DDD
PDF Exporter	Non	Aucun test pour l'export PDF
Compleo Routes (Express)	Non	Aucun test d'intégration HTTP
Pages React	Non	Aucun test de composant

Taux de couverture estimé : environ **35-40%** du code applicatif est couvert par des tests. Les modules côté client (lib/) représentant 10 682 lignes n'ont aucun test unitaire.

7. Qualité du Code

7.1 Métriques de Complexité

Métrique	Valeur	Seuil recommandé	Verdict
Fichiers > 500 lignes ("God Files")	18	0	Critique
Fichiers > 1 000 lignes	7	0	Critique
Usages de <code>any</code> type	65	0	Warning
<code>console.log</code> en production	4	0	Minor
Catch blocks vides	1	0	Minor
TODO/FIXME dans le code	19	0	Info (intentionnels dans le code généré)

7.2 Fichiers “God Object” (> 1 000 lignes)

Fichier	Lignes	Responsabilité	Recommandation
<code>extended-generator.ts</code>	1 619	14 transformations différentes	Découper en 1 fichier par transformation
<code>ai-engine.ts</code>	1 594	83 règles + scoring	Séparer règles par catégorie
<code>ComponentShowcase.tsx</code>	1 437	Galerie de composants	Découper en sous-composants
<code>ProjectDetail.tsx</code>	1 360	Workspace complet	Extraire les onglets en composants
<code>legacy-analyzer.ts</code>	1 204	10 détecteurs de technologies	1 fichier par technologie
<code>spring-generator.ts</code>	1 184	25 fonctions de génération	Découper par type d'artefact
<code>Compleo.tsx</code>	921	4 étapes (upload, analyse, gen, résultats)	Extraire chaque étape

7.3 Principes SOLID

Principe	Respect	Détail
S — Single Responsibility	Partiel	Les fichiers > 1 000 lignes violent ce principe. <code>ProjectDetail.tsx</code> gère l'éditeur, l'analyse, la génération, les microservices, le cloud, et l'export.
O — Open/Closed	Bon	Les règles AI sont dans un tableau extensible. Les détecteurs de technologies sont modulaires.
L — Liskov Substitution	N/A	Pas d'héritage significatif dans le code TypeScript.
I — Interface Segregation	Bon	Les interfaces IR (ProjectIR, UseCaseIR, DtoIR) sont bien séparées et spécifiques.
D — Dependency Inversion	Partiel	Les modules client importent directement les implémentations. Le serveur utilise l'injection via tRPC context.

7.4 Patterns et Anti-Patterns

Patterns positifs identifiés :

- Utilisation cohérente de TypeScript avec interfaces typées
- Séparation client/serveur via tRPC avec types partagés
- Système de règles extensible (tableau de règles avec pattern matching)
- IR (Intermediate Representation) comme contrat entre parser et générateur
- Web Workers pour le traitement parallèle

Anti-patterns identifiés :

- **God Objects** : 7 fichiers dépassant 1 000 lignes
- **Feature Envy** : `ProjectDetail.tsx` accède directement à 6 modules d'analyse
- **Primitive Obsession** : 65 usages de `any` type au lieu de types stricts
- **Stockage volatile** : Les sessions Compleo sont en `Map<>` mémoire (perdues au redémarrage)

- **Absence d'authentification** : 25 procédures tRPC sont `publicProcedure` (0 `protectedProcedure`)

8. Sécurité

Risque	Sévérité	Détail
Toutes les procédures tRPC sont publiques	Critique	25 <code>publicProcedure</code> , 0 <code>protectedProcedure</code> . N'importe qui peut créer/supprimer des projets.
API Compleo sans authentification	Critique	Les 7 endpoints Express n'ont aucun middleware d'auth.
Upload ZIP sans validation de taille côté serveur	Élevé	Multer limite à 50 Mo, mais pas de validation du contenu (zip bomb potentiel).
Sessions Compleo en mémoire	Moyen	Pas de persistance. Vulnérable au DoS par création massive de sessions.
Pas de rate limiting	Moyen	Aucun middleware de rate limiting sur les endpoints.
Pas de CORS restrictif	Faible	CORS géré par le proxy Manus, mais pas de configuration explicite.

9. Performance

Aspect	État	Détail
Analyse parallèle (Web Workers)	Bon	Distribution adaptative sur N cœurs CPU
Bundle size	Non optimisé	Cytoscape.js, jsPDF, xlsx, Recharts chargés globalement (pas de lazy loading)
Code splitting	Absent	Toutes les pages sont dans le bundle principal
Lazy loading des routes	Absent	Toutes les pages importées statiquement dans App.tsx
Requêtes DB	Correct	Drizzle ORM avec requêtes simples, pas de N+1 détecté
Compleo parsing	Bon	78 fichiers Java parsés en < 100ms côté serveur

10. Accessibilité et UX

Aspect	Score	Détail
Attributs ARIA	8 occurrences	Insuffisant pour une application de cette taille
Breakpoints responsive	15 occurrences	Couverture minimale, certaines pages non responsive
États de chargement	37 occurrences	Bonne couverture des loading states
Gestion d'erreurs UI	53 occurrences	Bonne couverture des error states
Thème sombre	Complet	Design "Terminal Craft" cohérent
Navigation clavier	Non vérifié	Pas de tests d'accessibilité

11. Dépendances

Catégorie	Nombre	Remarque
Production	73	Élevé. Certaines potentiellement inutilisées (embla-carousel, cmdk, input-otp).
Développement	27	Standard pour un projet React + Vite + Vitest.
Vulnérabilités connues	Non vérifié	<code>pnpm audit</code> non exécuté.

12. Recommandations Priorisées

Priorité 1 — Critique (à corriger immédiatement)

1. **Activer l'authentification** : Remplacer `publicProcedure` par `protectedProcedure` sur toutes les mutations (create, update, delete). Garder les queries de lecture en public si nécessaire.
2. **Persister les sessions Compleo** : Migrer le `Map<>` en mémoire vers la base de données MySQL (nouvelle table `compleo_sessions`).
3. **Ajouter du rate limiting** : Installer `express-rate-limit` sur les endpoints Compleo (upload, analyze, generate).

Priorité 2 — Élevée (sprint suivant)

1. **Découper les God Objects** : Refactorer les 7 fichiers > 1 000 lignes en modules plus petits (< 300 lignes chacun).
2. **Ajouter des tests pour les modules client** : Couvrir `ai-engine.ts` (83 règles), `legacy-analyzer.ts` (10 technologies), et `ejb-analyzer.ts` avec des tests unitaires. Objectif : 70% de couverture.
3. **Éliminer les any types** : Remplacer les 65 usages de `any` par des types stricts.

4. **Ajouter le lazy loading** : Utiliser `React.lazy()` + `Suspense` pour les routes dans `App.tsx`.

Priorité 3 — Moyenne (backlog)

1. **Validation du contenu ZIP** : Vérifier la taille décompressée et le nombre de fichiers avant extraction.
 2. **Ajouter des tests d'intégration** : Tester les endpoints Compleo Express avec supertest.
 3. **Améliorer l'accessibilité** : Ajouter des attributs ARIA, des labels, et tester avec un lecteur d'écran.
 4. **Optimiser le bundle** : Analyser avec `vite-bundle-analyzer` et tree-shake les imports inutilisés.
 5. **Ajouter un système de logging structuré** : Remplacer les `console.log` par un logger (winston ou pino).
-

13. Métriques Globales

Métrique	Valeur
Lignes de code source	29 970
Lignes de tests	1 710
Ratio tests/code	5.7%
Fichiers source	123
Fichiers de tests	5
Tests unitaires	119
Assertions	243
Durée des tests	791ms
Pages frontend	10
Procédures tRPC	25
Endpoints REST	7
Tables DB	7 (+1 users)
Règles IA	83
Technologies détectées	10
Transformations supportées	8
Dépendances production	73
Dépendances dev	27
God Files (> 1 000 lignes)	7
Usages <code>any</code>	65
Couverture de tests estimée	35-40%

14. Conclusion

La plateforme Java Legacy Modernizer v4.0 est un produit fonctionnel et ambitieux qui couvre un périmètre large : analyse multi-technologies (10 technologies, 83 règles), génération de code (8 transformations + Compleo), visualisation d'architecture (Cytoscape.js), simulation de migration (Strangler Fig), collaboration (commentaires, partage), et export (PDF, Excel, ZIP). Le moteur Compleo côté serveur est particulièrement abouti avec un pipeline complet EJB → Spring Boot testé sur 3 projets réels.

Les axes d'amélioration principaux sont la sécurité (authentification absente), la maintenabilité (God Objects), et la couverture de tests (35-40%). Les recommandations de ce rapport sont classées par priorité pour guider les prochains sprints de développement.

Rapport généré le 8 avril 2026 — Hamza NORDINE